Why is Reranking Required

ReRanking

Ranks → shuffle. re-order

Why it is required?

What is reranking?

How it is applied?

Ensemble Retriever - Reranking ✓ RRF (Reciprocal Rank fusion)

RAG Pipeline

Knowledge source → Knowledge Base.

① Ingestion Phase

② Retrieval Phase.

Raw docs → Doc loader + Text splitter → Embedding model → Encoding ✗ Knowledge | Info compression.

Text chunk → 1000 char → 256 dims

Text → Vectors

Semantic Meaning

Imp info.

Dense Representation

fine grained info → lossy compression.

Embedding vector → Decoder → Text chunk

Retrieve

Vector stores → Persist (store) + Indexing

Embedding model

Retrieval Phase.

Text Query → (Embedding model) → Query Vector (256 dim) → VS

Text Query → (Semantic meaning) → Query Vector

VS → (Semantic search) → Semantic similarity operation

Query vector + doc embedding → Similarity Calculate → Sim Score.

Comparison b/w semantic meaning of I/P and Docs

Sim Score → (Most similar) → ① Rank 1, ② Rank 2, ③ Rank 3

Sim Score → Sorting (Descending Order) → Rank.

Most similar → 10

Rank → Top k docs return.

Top Ranking docs

k:3, k:5, k:10

Re Ranking

After Retrieval

Initial Ranks → Ranking

Vectors → Info approximate

Doc embeddings → lossy compression.

Query vector → lossy compression

} similarity

Doc → Higher score → Relevant ✗ + noise.

Imp detail ✗

Doc → lower score. → lossy.

Similarity search ↓

lower Rank

Topk docs. k:3

Re Ranking

Embedding model ✗

→ Numbers.

→ Semantic Meaning

Lossy compression.

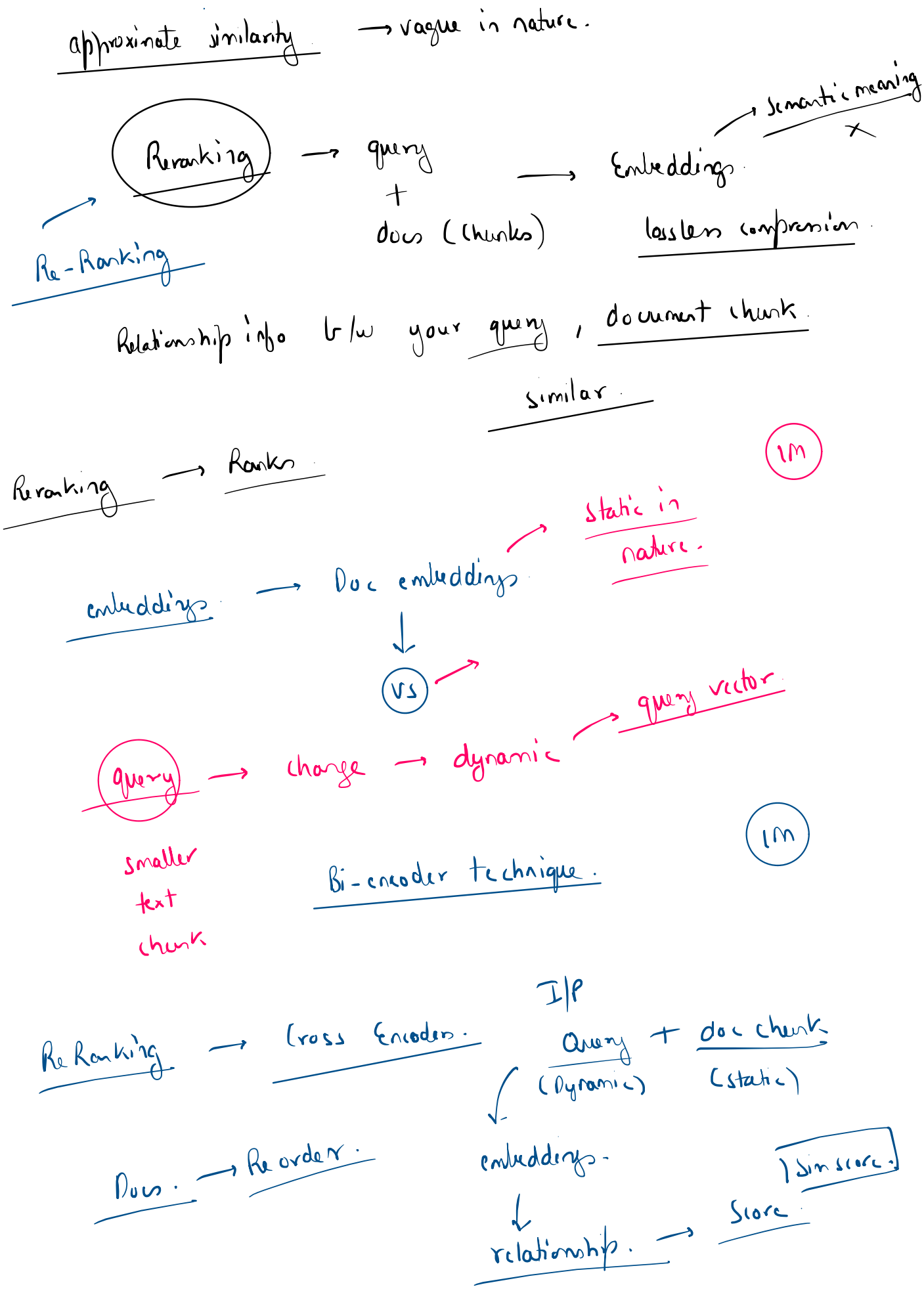
Scores. → Comparison. → lossy ✗

Compression.

fine grained info

Relevant doc → Rank I

Context Build. → Valuable info without noise.


approximate similarity → vague in nature.
Reranking → query + docs (chunks) → Embeddings → Semantic meaning ✗
Re-Ranking
lossless compression
Relationship info b/w your query, document chunk
similar
Reranking → Ranks
1M
embeddings → Doc embeddings → static in nature.
VS
query → change → dynamic → query vector
smaller text chunk
Bi-encoder technique.
1M
Re Ranking → Cross Encoders.
I/P
Query + doc chunk
(Dynamic) (static)
embeddings.
Docs. → Reorder.
relationship. → Score
Sim score.

Bi encoder technique.

query → encode.
docs → encode.
} faster

approximate encode.

Cross encoder technique

query + doc → dynamic → encode } slower.

Retrival.

↓ slow in operation

Re Ranking

→ filtering → Bi encoder → K: 5

→ Re-Ranking → Cross encoder

K = 20 → 20 similar docs → approximate similarity → noise discard

ordering ↓

Reorder.

Re Ranking

20 docs → Re Ranker

Doc + query → New Score.

New score ↑

sorting Desc.

K = 5

1m X

20

query ↑

Doc → static → VS } fast Bi encoder

Query → Dynamic

Q

D

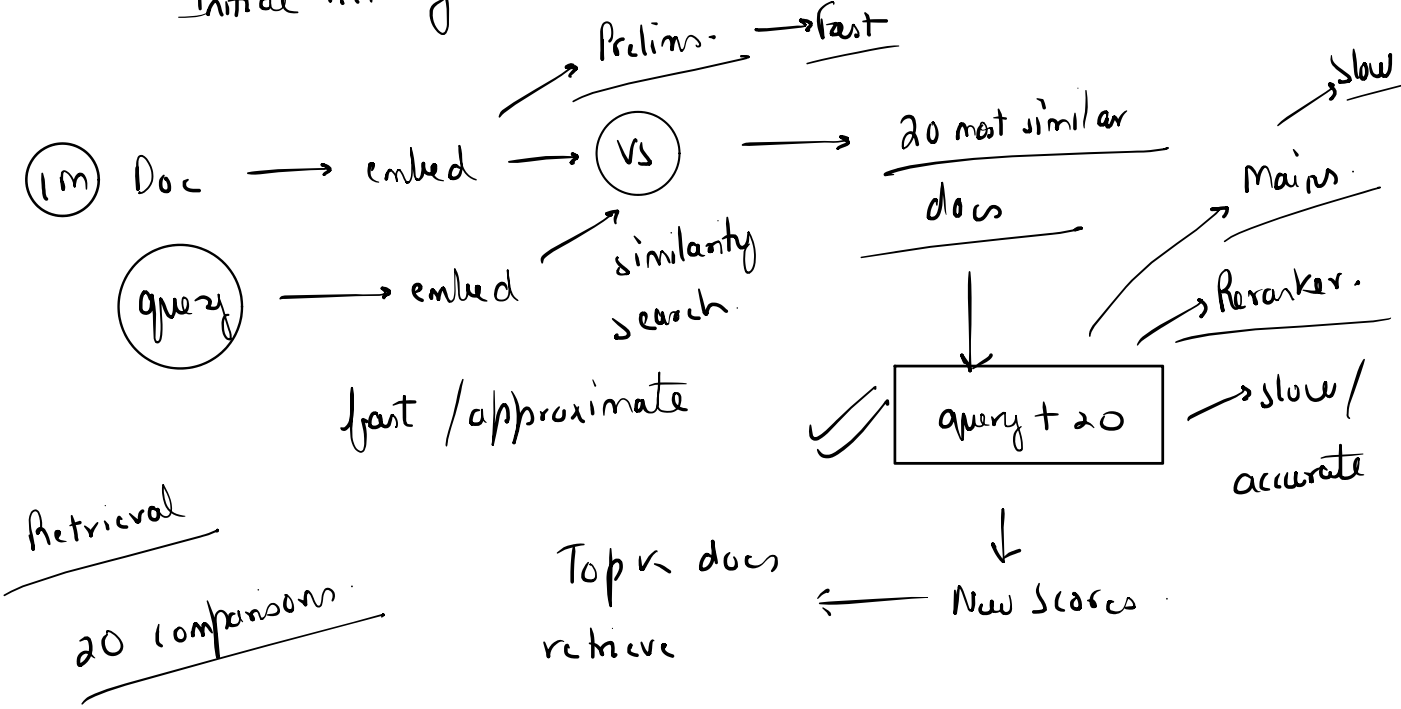
Query → Dynamic

↖ approximate

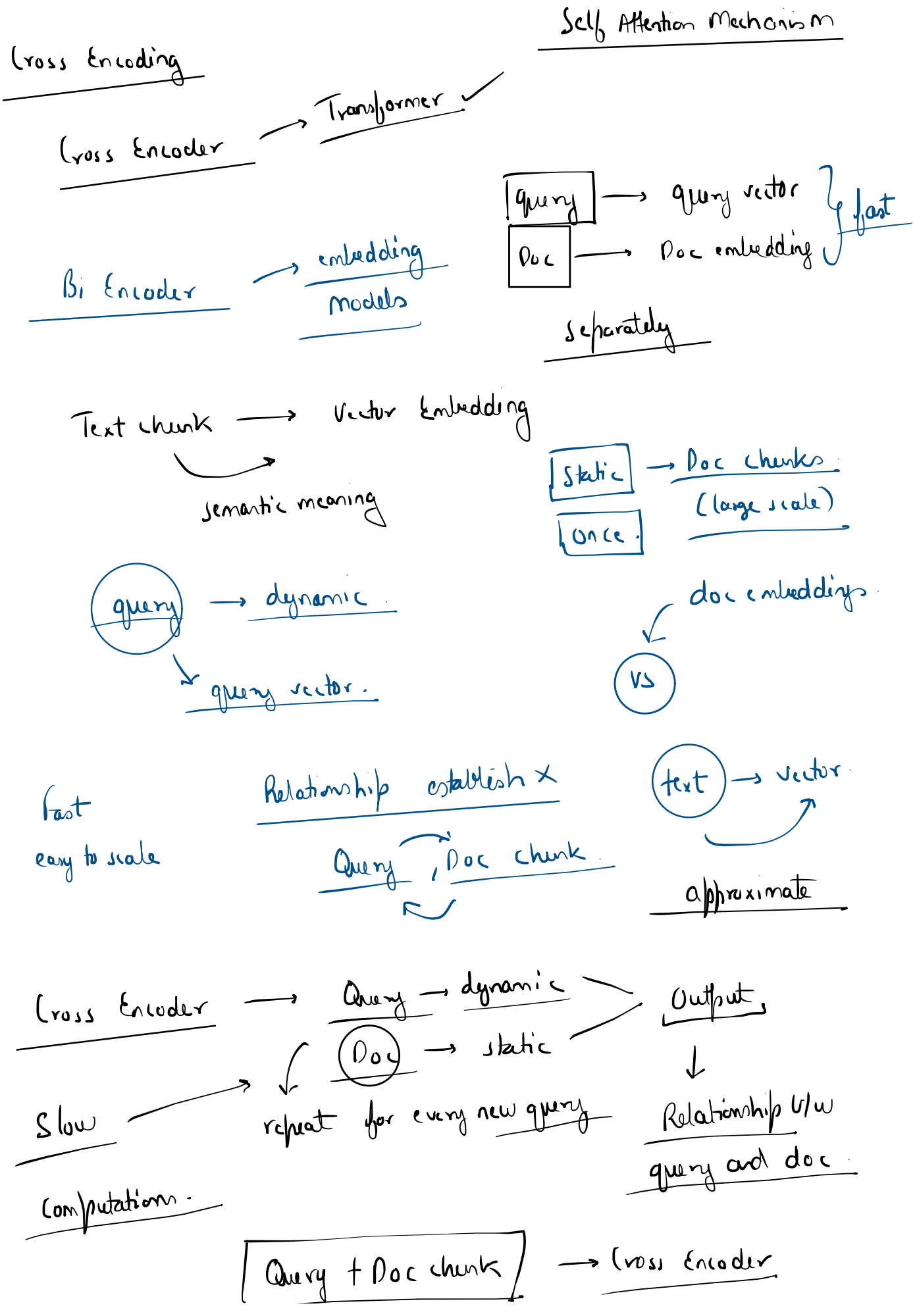
D

K: 5 J: 20

Initial filtering



How is it done



Query + Doc chunk → Cross Encoder

accurate

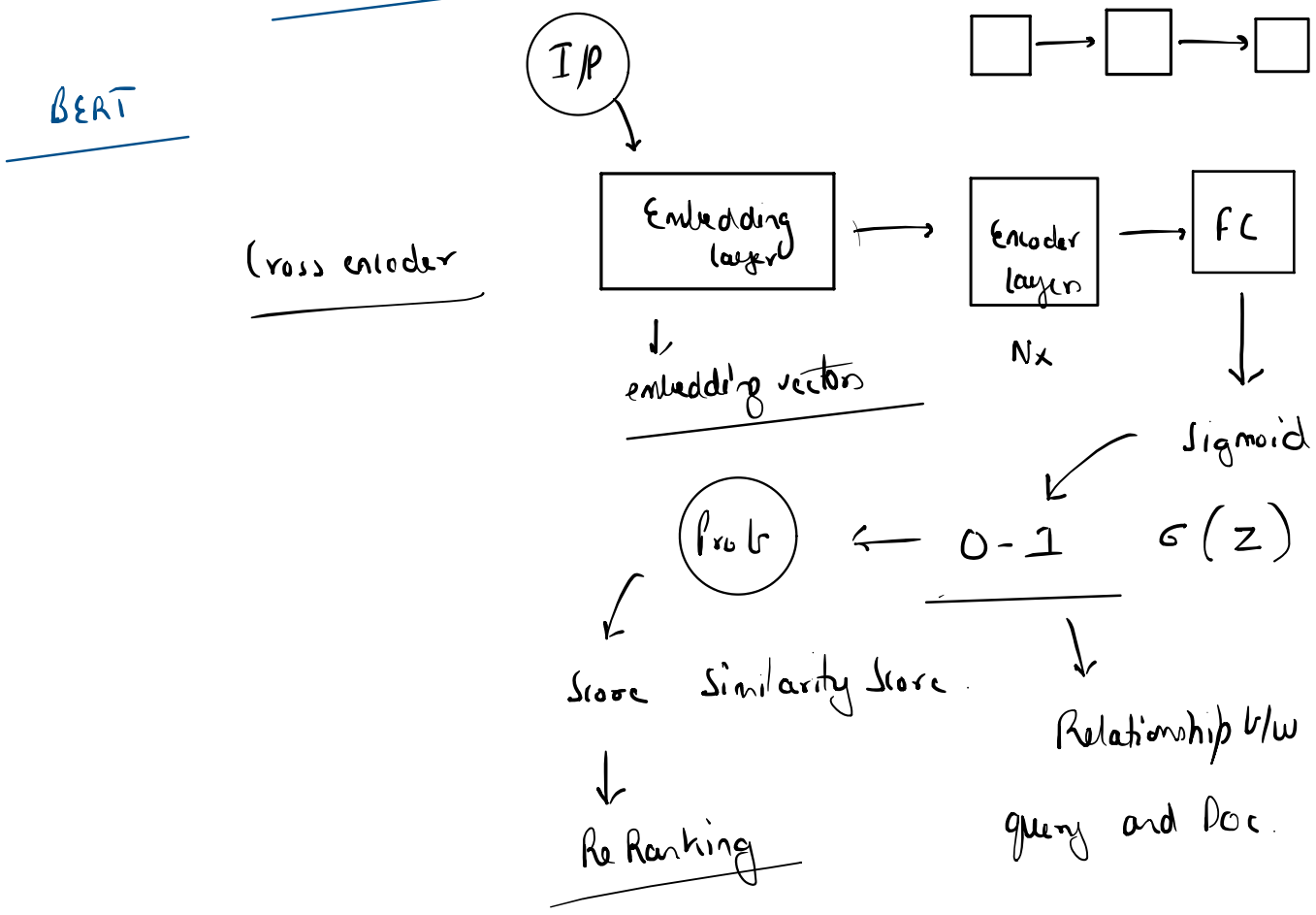
Cross Encoder → Transformer (Self Attention mechanism)

Input and output tokens

Relationship

Context aware embeddings

BERT



I/P Query + Doc Chunk

[CLS] Query [SEP] Doc Chunk [SEP]

① SEP → End of Query Tokens.

```
[CLS] query tokens [SEP] document tokens [SEP]
```

① Classification Token → Result ② SEP → end of Doc tokens.

② Separation Token

query → doc
doc → query
} Bi directional. architecture.

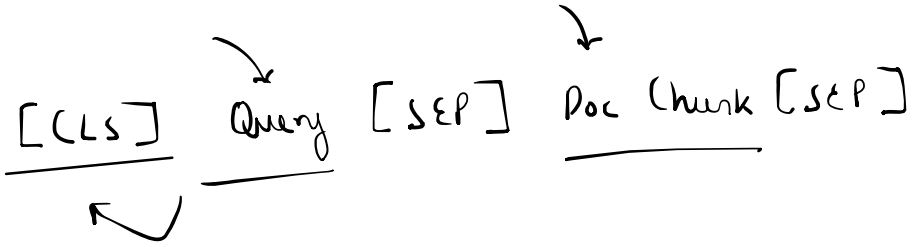
```
Input Sequence
      ↓
[Embedding Layer]  →  converts tokens to vectors
      ↓
[Transformer Layer 1]  →  every token attends to every other token
      ↓
[Transformer Layer 2]  →  richer, more contextual representations
      ↓
   ... (N layers)
      ↓
[Final Layer]
```

→ Self Attention.

```
[CLS] final vector  →  Linear Layer  →  Score (0 to 1)
```

→ 0 - 1

Embedding vector



CLS Aggregate

embedding layer → Embeddings. → Raw embeddings

Stack of encoder layer → Self Attention Mechanism.

Bi directional fashion.

query token attends to each doc token

Doc token attends to each query token.

detailed relationship. CNN. edges.

detailed relationship

CNN · edges + Patterns

embeddings → refine

linear layer → FC → Sigmoid

↓

Relevance Score → New Score

CLS → vector

Relationship

Query + Doc

10 tokens

↓

10 vector

300 token

↓

300 vector

CLS → update → Relationship

313 vector ← 2 SEP

1 CLS

$\sigma(z)$ = Score

CLS

embeddings → update relationship

Self Attention

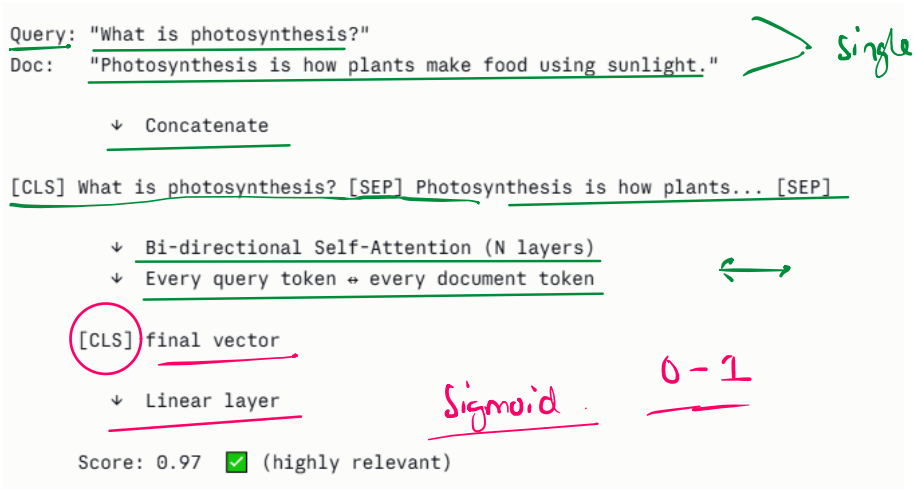
↓

Updated vectors

[CLS] →

Self Attention

```
Layer 1:  [CLS] attends to raw token embeddings
              ↓
Layer 2:  [CLS] attends to Layer-1-enriched vectors
              ↓
Layer 3:  [CLS] attends to Layer-2-enriched vectors
              ↓
  ...   (12 layers in BERT-base)
              ↓
Layer 12: [CLS] has now accumulated context from all tokens,
          processed through 12 rounds of deep mixing
```



> single input (Query + Doc)

↔ → 12 layers.

Sigmoid 0 - 1

Linear layer → Sigmoid → Score

CLS → Start of the input

token → relationship.

BERT → training

Accumulate

Encoder layers -

[CLS] → vector → update

query → doc

Self Attention → token, all the other token.

Approximate filtering → Bi encoder → Sim search } fast approx.

Deep Refinement → Reranking → Cross encoder

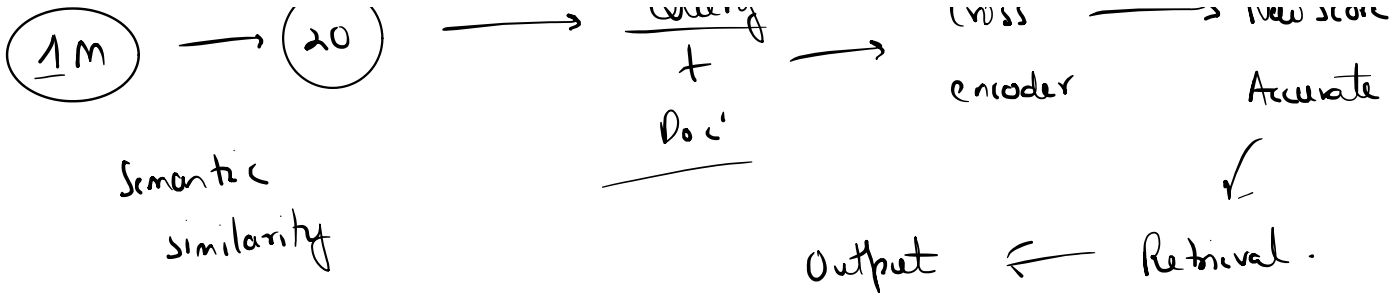
Query + Doc } computations slow

Subsets of Docs

Accurate

1M → 20 → Query + → Cross encoder → New Score Accurate

Slow


1M
20
+
Doc
encoder
Accurate
Semantic
similarity
Output
Retrival.

Code